

House Prices using Backward Elimination

importing all the libraries

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

%matplotlib inline
```

importing dataset using panda

```
In [3]: dataset = pd.read_csv(r'C:\Users\sande\OneDrive\Desktop\Naresh IT\data science and dataset')
```

```
Out[3]:
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080
...	...	...	...	...	...	...	...
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076

21613 rows × 21 columns



```
In [4]: dataset.head()
```

Out[4]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	flo
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	

5 rows × 21 columns



checking if any value is missing

In [5]: `print(dataset.isnull().any())`

```
id                False
date              False
price             False
bedrooms          False
bathrooms         False
sqft_living       False
sqft_lot          False
floors            False
waterfront        False
view              False
condition         False
grade             False
sqft_above        False
sqft_basement     False
yr_built          False
yr_renovated      False
zipcode           False
lat               False
long              False
sqft_living15     False
sqft_lot15        False
dtype: bool
```

checking for categorical data

In [6]: `print(dataset.dtypes)`

```

id                int64
date              object
price             float64
bedrooms          int64
bathrooms         float64
sqft_living       int64
sqft_lot          int64
floors            float64
waterfront        int64
view              int64
condition         int64
grade             int64
sqft_above        int64
sqft_basement     int64
yr_built          int64
yr_renovated      int64
zipcode           int64
lat               float64
long              float64
sqft_living15     int64
sqft_lot15        int64
dtype: object

```

dropping the id and date column

```
In [7]: dataset = dataset.drop(['id', 'date'], axis = 1)
```

understanding the distribution with seaborn

```
In [8]: with sns.plotting_context("notebook", font_scale=2.5):
        g = sns.pairplot(dataset[['sqft_lot', 'sqft_above', 'price', 'sqft_living', 'bedrooms'],
                                hue='bedrooms', palette='tab20', size=6)
        g.set(xticklabels=[]);
```

C:\Users\sande\anaconda3\Lib\site-packages\seaborn\axisgrid.py:2100: UserWarning: The `size` parameter has been renamed to `height`; please update your code.
warnings.warn(msg, UserWarning)

separating independent and dependent variable

```
In [10]: X = dataset.iloc[:,1:].values
        y = dataset.iloc[:,0].values
        #splitting dataset into training and testing dataset
        from sklearn.model_selection import train_test_split
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 1/3, random_s
```

```
In [11]: from sklearn.linear_model import LinearRegression
        regressor = LinearRegression()
        regressor.fit(X_train, y_train)
```

```
# Predicting the Test set results
y_pred = regressor.predict(X_test)
```

Backward Elimination

```
In [15]: import numpy as np
import statsmodels.api as sm

def backwardElimination(X, y, SL=0.05):
    # Ensure inputs are NumPy arrays with correct shape
    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float).reshape(-1, 1)

    # Add intercept term
    X = sm.add_constant(X)

    numVars = X.shape[1]
    selected_vars = list(range(numVars))

    while True:
        X_opt = X[:, selected_vars]
        regressor_OLS = sm.OLS(y, X_opt).fit()
        p_values = regressor_OLS.pvalues

        max_p_value = max(p_values)
        if max_p_value > SL:
            max_p_index = np.argmax(p_values)
            del selected_vars[max_p_index]
        else:
            break

    print(regressor_OLS.summary())
    return X[:, selected_vars]

# Example usage:
# Assuming X and y are already defined and preprocessed
SL = 0.05
X_modeled = backwardElimination(X, y, SL)
```

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          0.700
Model:                  OLS    Adj. R-squared:       0.699
Method:                 Least Squares  F-statistic:       3145.
Date:                   Wed, 17 Sep 2025  Prob (F-statistic): 0.00
Time:                   01:21:25  Log-Likelihood:    -2.9460e+05
No. Observations:      21613    AIC:               5.892e+05
Df Residuals:          21596    BIC:               5.894e+05
Df Model:               16
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	5.741e+06	2.89e+06	1.989	0.047	8.28e+04	1.14e+07
x1	-3.586e+04	1891.248	-18.962	0.000	-3.96e+04	-3.22e+04
x2	4.272e+04	3141.958	13.596	0.000	3.66e+04	4.89e+04
x3	109.9730	2.256	48.752	0.000	105.551	114.394
x4	0.1266	0.048	2.643	0.008	0.033	0.221
x5	5.831e+05	1.74e+04	33.585	0.000	5.49e+05	6.17e+05
x6	5.297e+04	2139.565	24.756	0.000	4.88e+04	5.72e+04
x7	2.614e+04	2347.831	11.133	0.000	2.15e+04	3.07e+04
x8	9.624e+04	2144.551	44.878	0.000	9.2e+04	1e+05
x9	72.3485	2.091	34.593	0.000	68.249	76.448
x10	37.6313	2.412	15.604	0.000	32.904	42.358
x11	-2590.7927	70.920	-36.531	0.000	-2729.801	-2451.784
x12	20.1729	3.651	5.526	0.000	13.017	27.328
x13	-576.6895	32.844	-17.559	0.000	-641.065	-512.314
x14	6.044e+05	1.07e+04	56.494	0.000	5.83e+05	6.25e+05
x15	-2.168e+05	1.31e+04	-16.568	0.000	-2.42e+05	-1.91e+05
x16	20.9673	3.426	6.119	0.000	14.251	27.683
x17	-0.3874	0.073	-5.291	0.000	-0.531	-0.244

```

=====
Omnibus:                18333.238  Durbin-Watson:          1.990
Prob(Omnibus):           0.000    Jarque-Bera (JB):        1848729.005
Skew:                    3.552    Prob(JB):                 0.00
Kurtosis:                47.749    Cond. No.                 2.68e+17
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 3.05e-21. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.